Robust Quaternion Estimation with Geometric Algebra

Mauricio Cele Lopez Belon

Abstract. Robust methods for finding the best rotation aligning two sets of corresponding vectors are formulated in the linear algebra framework, using tools like the SVD for polar decomposition or QR for finding eigenvectors. Those are well established numerical algorithms which on the other hand are iterative and computationally expensive. Closed form solutions have been proposed in the quaternion framework; they are fast, but they have singularities, i.e., they completely fail on certain input data. In this paper we formulate the problem in Geometric Algebra and follow the geometry at every step. The best rotor maximizes an energy that is quadratic in the rotor, and its multivector derivative leads to an eigenrotor equation. Two energies that are sums of squares bound the eigenvalues, and the bound is a starting point from which Newton's method provably converges to the largest eigenvalue, in one step when the data are free of noise. The eigenrotor is then found as the intersection of hyperplanes, computed with outer products in the 4D Geometric Algebra. This geometric view explains the singularities of the closed form methods, and of the tempting alternative of combining all the hyperplanes symmetrically, as a single phenomenon: they all fix in advance the scale of one hyperplane, or of one combination of them, and every such choice fails on a whole family of rotations. Choosing the hyperplanes from the data removes the singularities, without any data-dependent branch. The method reaches the optimum on all our test cases, with a speed comparable to the fastest methods for small numbers of vectors; when several problems are solved at once with SIMD instructions, it is the fastest of the methods we tested that return a quaternion at full accuracy, up to 29% faster than QUEST.

Mathematics Subject Classification (2010). Primary 15A66; Secondary 65F15.

Keywords. Geometric Algebra, Quaternion Estimation, Wahba Problem.

1. Introduction

The estimation of rotations has been studied for over half a century [19]. The problem consists of finding the optimal rotation aligning two sets of corresponding vectors, and it appears in spacecraft attitude determination, point cloud registration, computer vision and shape deformation. Many effective methods have been developed [1,9,14,17,23] using 3×3 matrices and quaternions. Formulations based on quaternions solve a max-eigenvalue problem, while formulations based on linear algebra rely on the Singular Value Decomposition (SVD). Formulations based on geometric algebra [5,15] were introduced in the last decades, but they also rely on linear algebra numerical algorithms such as the SVD. Closed form solutions for finding the optimal quaternion have been proposed [20–22,24], based on analytic formulas for the roots of the quartic polynomial associated with the eigenvalue problem.

Accuracy and speed of prominent methods have been compared in [6,13,22], evidencing a trade-off between performance and robustness. SVD based methods exhibit the best accuracy but low performance, and quaternion based methods are faster but less robust. The closed form solutions exhibit the best performance so far, but they have singularities, i.e., they completely fail on certain input data. QUEST [17], for instance, fails for every rotation by π, and FLAE [22] fails for every rotation whose axis is orthogonal to the z axis, including the identity.

In this paper we formulate the problem in the $\mathbb{G}_3$ geometric algebra, where rotations are rotors, and we try to understand each step geometrically, not only to compute it. Rotors are isomorphic to quaternions, but geometric algebra treats rotations and the subspaces of $\mathbb{R}^3$ in the same manner, and this facilitates meaningful algebraic manipulations. We work with bivectors, planes through the origin, because they belong to the same subalgebra as rotors, so that all the products we need stay in that subalgebra; by the duality of vectors and bivectors in $\mathbb{G}_3$, everything holds for vectors as well. The paper follows the path from the problem to the algorithm:

- We write an energy that is maximal when the rotated bivectors coincide with their targets, show that it is quadratic in the rotor, and derive the eigenrotor equation from its multivector derivative (Section 3).
- Two energies that are sums of squares bound the eigenvalues. The bound tells us where to start Newton's method so that it converges to the largest eigenvalue, and it is exact when the data are free of noise (Sections 4 and 5).
- In the 4D geometric algebra, the eigenrotor is the line where four hyperplanes meet, and the meet of any three of them is computed with outer products (Section 6). We show how large each of these meets is, which explains the singularities of the closed form methods and how to avoid them (Section 8).

Section 9 gives the algorithm and Section 10 compares it with other methods, including a comparison with QUEST, FOAM and FLAE when many problems are solved in parallel with SIMD instructions, where the absence

of data-dependent branches in our method pays off. Section 11 applies the method to the rotation fits of smooth-rotation ARAP mesh deformation.

Two natural ideas turn out to be misleading, and we discuss both because seeing why they fail is the clearest way to understand what works. The first is that, since every single hyperplane fails for some rotations, constraining all of them at once, symmetrically, should avoid every failure. As we show in Section 6.4, that symmetric choice is itself a fixed choice, and it fails, for example, for every rotation by $\pi/2$ about the z axis. The second is that a positive semi-definite energy makes Newton's method robust. Positivity is useful, but what makes the iteration robust is where it starts, as explained in Section 5.

2. Geometric Algebra $\mathbb{G}_3$

A geometric algebra $\mathbb{G}_3$ is constructed over a real vector space $\mathbb{R}^3$, with basis vectors $\{e_1, e_2, e_3\}$. The associative geometric product is defined on vectors so that the square of any vector a is a scalar, $aa = a^2 = |a|^2$, and the geometric product of two vectors a and b is

$$ab = a \cdot b + a \wedge b, \qquad ba = a \cdot b - a \wedge b. \tag{2.1}$$

The symmetric part $a \cdot b = \frac{1}{2}(ab + ba)$ is the usual inner product, a scalar. The antisymmetric part $a \wedge b = \frac{1}{2}(ab - ba)$ is a new algebraic entity, a *bivector* or 2-vector, which can be visualized as an oriented plane element: the two-dimensional analogue of a direction. In particular, orthogonal vectors anticommute, $e_i e_j = -e_j e_i$ for $i \neq j$. Bivectors can be decomposed in the basis $\{e_{23}, e_{31}, e_{12}\}$, where $e_{ij} = e_i e_j = e_i \wedge e_j$.

The outer product of three vectors $a \wedge b \wedge c$ is a 3-vector, also known as the pseudoscalar, because the trivector basis consists of the single element $I = e_1 e_2 e_3$. Scalars are 0-vectors. The outer product of k vectors is a k-vector, k is its *grade*, and it vanishes if and only if the vectors are linearly dependent. Every multivector X is a sum of parts of different grades, and $\langle X \rangle_k$ denotes the part of grade k; we write $\langle X \rangle = \langle X \rangle_0$ for the scalar part. The pseudoscalar commutes with every element of $\mathbb{G}_3$ and $I^2 = -1$. Multiplying by I maps every vector to the bivector of the plane orthogonal to it, and conversely: this is the *duality* of vectors and bivectors. For two vectors, the bivector of their plane is dual to their cross product,

$$a \wedge b = I\,(a \times b). \tag{2.2}$$

Reversing the order of multiplication is called reversion: $\widetilde{(a_1 a_2 \cdots a_k)} = a_k \cdots a_2 a_1$. Scalars and vectors are their own reverses, while bivectors and trivectors change sign, and $\widetilde{(XY)} = \widetilde{Y}\widetilde{X}$. Two properties of the scalar part will be used constantly. The scalar part of a product does not depend on the order of the two factors, and reversion does not change it:

$$\langle XY \rangle = \langle YX \rangle, \qquad \langle \widetilde{X} \rangle = \langle X \rangle. \tag{2.3}$$

The first property implies that the factors of a product inside $\langle\,\rangle$ can be permuted cyclically, as in the trace of a product of matrices.

2.1. Rotors

Rotations are even grade multivectors known as rotors. We denote the subalgebra of even multivectors, those of grades 0 and 2, as $\mathbb{G}_3^+$; it has dimension four. A reflection of any k-vector X in a plane with normal n is expressed as the sandwich product $(-1)^k nXn$, and a rotation is the composition of two reflections. The most basic rotor R is therefore the product of two unit vectors a and b at an angle of $\frac{\theta}{2}$: reflecting in the plane orthogonal to b and then in the one orthogonal to a rotates by θ in the plane of a and b. With the unit bivector $B = \frac{a\wedge b}{|a\wedge b|}$,

$$ab = a\cdot b + a\wedge b = \cos\left(\frac{\theta}{2}\right) + B\sin\left(\frac{\theta}{2}\right). \tag{2.4}$$

Rotors act on all k-vectors with the sandwich product $X' = RX\widetilde{R}$, where $\widetilde{R}$ is the reverse of R. For a rotor, $R\widetilde{R} = 1$.

It is useful to see rotors as vectors of a four-dimensional Euclidean space. We equip $\mathbb{G}_3^+$ with the scalar product and norm

$$X * Y = \langle X\widetilde{Y}\rangle, \qquad |X|^2 = X * X. \tag{2.5}$$

For $X = w + B$, with a scalar w and a bivector $B = Iv$, we have $\widetilde{X} = w - B$ and $X\widetilde{X} = w^2 - B^2 = w^2 + |v|^2$, because $B^2 = I^2v^2 = -|v|^2$. So $X\widetilde{X} = |X|^2$ is always a scalar, and *the rotors are exactly the elements of* $\mathbb{G}_3^+$ *of norm one*: they form the unit sphere S^3 of a four-dimensional Euclidean space. This simple fact will matter: whatever element of $\mathbb{G}_3^+$ we compute, dividing it by its norm makes it a rotor, and R and $-R$ represent the same rotation.

2.2. Bivector products

The commutator product of two bivectors p and q is $p\times q = \frac{1}{2}(pq - qp)$. In $\mathbb{G}_3$ it can be interpreted as a cross product of bivectors: the bivector $p\times q$ is orthogonal to both p and q. It allows us to write the geometric product of two bivectors as $pq = p\cdot q + p\times q$, a scalar plus a bivector. The inner product of bivectors differs from the inner product of vectors in the sign: since the square of a bivector is negative, the inner product of bivectors is a negative scalar, e.g., $(ae_{12} + be_{13} + ce_{23})\cdot(de_{12} + ee_{13} + fe_{23}) = -ad - be - cf$. With the scalar product (2.5), instead, $p*q = \langle p\widetilde{q}\rangle = -p\cdot q$ is positive for a bivector and itself.

2.3. Quaternions

A quaternion $w + x\mathbf{i} + y\mathbf{j} + z\mathbf{k}$ consists of a scalar part w and a vector part, in a basis of imaginary units that square to -1 and anticommute: $\mathbf{i}^2 = \mathbf{j}^2 = \mathbf{k}^2 = \mathbf{ijk} = -1$. In geometric algebra they correspond to bivectors. We choose the basis of $\mathbb{G}_3^+$

$$E_0 = 1, \qquad E_1 = e_3e_2, \qquad E_2 = e_1e_3, \qquad E_3 = e_2e_1, \tag{2.6}$$

that is, $E_k = -Ie_k$, the planes orthogonal to the basis vectors, with the orientation that makes them multiply like Hamilton's units: $E_k^2 = -1$ and $E_1E_2 = (-Ie_1)(-Ie_2) = I^2e_1e_2 = -e_{12} = E_3$. The basis is orthonormal for the scalar product (2.5). A rotor $R = w + xE_1 + yE_2 + zE_3$ corresponds to the quaternion $w + x\mathbf{i} + y\mathbf{j} + z\mathbf{k}$, and we call (w, x, y, z) its coordinates. With this orientation, the rotor of a rotation by θ about the unit axis n is [4]

$$R = \cos\frac{\theta}{2} - \sin\frac{\theta}{2}\,In = \cos\frac{\theta}{2} + \sin\frac{\theta}{2}\,(n_1E_1 + n_2E_2 + n_3E_3), \tag{2.7}$$

so its coordinates are exactly the unit quaternion of the same rotation. Note that the scalar part of a rotor is $\cos\frac{\theta}{2}$, which vanishes for the rotations by π, and its bivector part is proportional to $\sin\frac{\theta}{2}$, which vanishes for the identity. These two facts are behind the singularities of Section 8.

3. Geometric Algebra Rotor Estimation

3.1. The energy

We are given two sets of n corresponding bivectors $P = \{p_j\}_{j=1}^n$ and $Q = \{q_j\}_{j=1}^n$, and positive weights c_j. We want the rotor R that rotates each p_j as close as possible to q_j. The rotor R rotates p_j to q_j exactly when $Rp_j\widetilde{R} = q_j$, or equivalently, applying the inverse rotation $\widetilde{R}\,(\cdot)\,R$ to both sides, when $p_j = \widetilde{R}q_jR$. So we can either make the difference $p_j - \widetilde{R}q_jR$ as short as possible, or make the sum $p_j + \widetilde{R}q_jR$ as long as possible. The two are the same problem, because of the parallelogram law,

$$|p + \widetilde{R}qR|^2 + |p - \widetilde{R}qR|^2 = 2\,|p|^2 + 2\,|q|^2, \tag{3.1}$$

which does not depend on R (a rotation does not change lengths). The sum is longest exactly when the difference is shortest. We choose the sum and maximize the energy

$$E(R) = \sum_j c_j\,|p_j + \widetilde{R}q_jR|^2, \qquad R\widetilde{R} = 1. \tag{3.2}$$

The weights need not sum to one. By (3.1), maximizing E is the same as minimizing the classical least squares error $\sum_j c_j|Rp_j\widetilde{R} - q_j|^2$ of Wahba's problem [19].

3.2. From quartic to quadratic

As written, the energy is hard to work with: $\widetilde{R}q_jR$ contains the rotor twice, so the squared norm contains it four times. The way out is to multiply by R on the left. For a rotor, $R\widetilde{R} = 1$, so

$$R\,(p_j + \widetilde{R}q_jR) = Rp_j + q_jR, \tag{3.3}$$

and multiplying by a rotor does not change the norm of an even multivector, $|RX| = |X|$, because $|RX|^2 = \langle RX\widetilde{X}\widetilde{R}\rangle = |X|^2\langle R\widetilde{R}\rangle$. Hence

$$E(R) = \sum_j c_j \, |Rp_j + q_j R|^2. \tag{3.4}$$

Now each term $Rp_j + q_j R$ is *linear* in R, so the energy is *quadratic* in R. This is what will turn the search for the best rotor into an eigenvalue problem. Note that the new form (3.4) makes sense for any element R of $\mathbb{G}_3^+$, not only for rotors, and that is how we will treat it from now on: as a quadratic function on the four-dimensional space $\mathbb{G}_3^+$, to be maximized on its unit sphere.

3.3. The multivector derivative

To find the maximum we take the multivector derivative of E with respect to R [8]. Restricted to $\mathbb{G}_3^+$, the derivative $\partial E(R)$ is the element of $\mathbb{G}_3^+$ such that, for every direction X,

$$\frac{d}{dt} E(R + tX)\Big|_{t=0} = X * \partial E(R). \tag{3.5}$$

It is the gradient of E for the scalar product (2.5). At a maximum of E on the unit sphere, the gradient must be orthogonal to the sphere, that is, parallel to R; this is the method of Lagrange multipliers, $\partial E(R) = \lambda \, \partial |R|^2 = 2\lambda R$.

First we expand the energy. Since p_j and q_j are bivectors, Rp_j and $q_j R$ are even, and $\widetilde{p_j} = -p_j$, $\widetilde{q_j} = -q_j$:

$$\begin{aligned} |Rp_j + q_j R|^2 &= \langle (Rp_j + q_j R)(\widetilde{p_j}\widetilde{R} + \widetilde{R}\widetilde{q_j})\rangle \\ &= -\langle Rp_j p_j \widetilde{R}\rangle - \langle Rp_j \widetilde{R} q_j\rangle - \langle q_j R p_j \widetilde{R}\rangle - \langle q_j R \widetilde{R} q_j\rangle. \end{aligned} \tag{3.6}$$

The first and last terms are the squared lengths of the bivectors times $|R|^2$: $p_j p_j = -|p_j|^2$, so $-\langle Rp_j p_j \widetilde{R}\rangle = |p_j|^2 |R|^2$, and similarly for q_j. The two middle terms are equal, by a cyclic permutation (2.3). Therefore

$$E(R) = S\,|R|^2 - 2\sum_j c_j \langle Rp_j \widetilde{R} q_j\rangle, \qquad S = \sum_j c_j \left(|p_j|^2 + |q_j|^2\right). \tag{3.7}$$

S is the total weighted energy of the data, and it does not depend on R. Everything that depends on the rotation is in the sum, so let us look at it more closely. Bringing R to the front with a cyclic permutation, $\langle Rp_j \widetilde{R} q_j\rangle = \langle R\,(p_j \widetilde{R} q_j)\rangle$, and the factor in parentheses is the reverse of $q_j R p_j$: $\widetilde{(q_j R p_j)} = \widetilde{p_j}\widetilde{R}\widetilde{q_j} = p_j \widetilde{R} q_j$. This suggests collecting the data into one linear function.

Definition 3.1. The *rotor operator* of the data is the linear function on $\mathbb{G}_3^+$

$$\mathcal{K}(R) = -\sum_j c_j \, q_j \, R \, p_j. \tag{3.8}$$

The product of a bivector, an even element and a bivector is even, so $\mathcal{K}$ maps $\mathbb{G}_3^+$ to itself. By the definition of the scalar product, $R * \mathcal{K}(R) = -\sum_j c_j \langle R\,\widetilde{(q_j R p_j)}\rangle = -\sum_j c_j \langle Rp_j \widetilde{R} q_j\rangle$, and the energy becomes

$$E(R) = S\,|R|^2 + 2\,R * \mathcal{K}(R). \tag{3.9}$$

The function $G(R) = R * \mathcal{K}(R)$ has a direct meaning. Since $\widetilde{q_j} = -q_j$, it can be written $G(R) = \sum_j c_j \, (Rp_j\widetilde{R}) * q_j$: it adds up, with the weights, how well each rotated bivector $Rp_j\widetilde{R}$ agrees with its target q_j. We call it the *gain*. The energy is the constant S plus twice the gain, so maximizing one maximizes the other.

$\mathcal{K}$ is symmetric, $X * \mathcal{K}(Y) = Y * \mathcal{K}(X)$. Indeed

$$X * \mathcal{K}(Y) = -\sum_j c_j \langle X \, \widetilde{(q_j Y p_j)} \rangle = -\sum_j c_j \langle X p_j \widetilde{Y} q_j \rangle;$$

reversing inside the scalar part gives $\langle q_j Y p_j \widetilde{X} \rangle$, and a cyclic permutation gives $\langle Y p_j \widetilde{X} q_j \rangle$, which is the same expression with X and Y exchanged. Because of the symmetry, the derivative of the quadratic function $R * \mathcal{K}(R)$ is $2\mathcal{K}(R)$: in $(R + tX) * \mathcal{K}(R + tX)$, the two terms linear in t are $X * \mathcal{K}(R)$ and $R * \mathcal{K}(X)$, which are equal. With $\partial |R|^2 = 2R$,

$$\partial E(R) = 2SR + 4\,\mathcal{K}(R) = 2 \sum_j c_j \left((|p_j|^2 + |q_j|^2)\, R - 2\, q_j R p_j \right), \qquad (3.10)$$

the same expression as the one obtained by taking the multivector derivative of each term of (3.6) directly. Setting it equal to $2\lambda R$ gives the *eigenrotor equation*

$$\mathcal{K}(R) = \mu\, R, \qquad \mu = \frac{\lambda - S}{2}. \qquad (3.11)$$

The best rotor is an eigenvector of $\mathcal{K}$. Which one? At a unit eigenrotor, the gain is $G(R) = R * \mu R = \mu$, so the best rotor is the eigenrotor of the *largest* eigenvalue μ_0, and the maximum energy is $S + 2\mu_0$. Because $\mathcal{K}$ is symmetric on a four-dimensional Euclidean space, it has four real eigenvalues with orthogonal eigenrotors (the spectral theorem), so this maximum exists and, as long as μ_0 is a simple eigenvalue, it is attained by a single rotation.

What is μ_0 when the data are free of noise? Then $q_j = Rp_j\widetilde{R}$, that is, $q_j R = Rp_j$ for the true rotor, and $\mathcal{K}(R) = -\sum_j c_j \, R\, p_j p_j = \sum_j c_j |p_j|^2 R$. Since rotations preserve lengths, $|p_j| = |q_j|$, and this is $\frac{S}{2}R$:

$$\mu_0 = \frac{S}{2} \qquad \text{for data free of noise.} \qquad (3.12)$$

We will see in Section 4 that $S/2$ is in fact an upper bound for all the eigenvalues, attained exactly when there is no noise, and in Section 5 that this is precisely what makes it the right place to start the search for μ_0.

3.4. The eigenproblem in coordinates

To compute, we need the matrix of $\mathcal{K}$ in the basis (2.6). It is easier to do the computation with vectors than with bivectors, since the product rules of vectors are simpler. Every bivector is the dual of a vector, $p_j = I\hat{p}_j$ and $q_j = I\hat{q}_j$, and since I commutes with everything and $I^2 = -1$,

$$\mathcal{K}(R) = -\sum_j c_j \, I\hat{q}_j \, R \, I\hat{p}_j = \sum_j c_j \, \hat{q}_j \, R \, \hat{p}_j. \qquad (3.13)$$

The data enter through two quantities: the correlation matrix and the vector of cross products,

$$D = \sum_j c_j\, \hat{q}_j \hat{p}_j^T, \qquad z = \sum_j c_j\, \hat{p}_j \times \hat{q}_j. \tag{3.14}$$

Let us apply $\mathcal{K}$ to the basis. For $R = 1$, by (2.1) and (2.2),

$$\mathcal{K}(1) = \sum_j c_j\, \hat{q}_j \hat{p}_j = \sum_j c_j\, (\hat{q}_j \cdot \hat{p}_j + \hat{q}_j \wedge \hat{p}_j) = \operatorname{tr} D - Iz,$$

whose coordinates are $(\operatorname{tr} D, z)$, since $-Iz = z_1 E_1 + z_2 E_2 + z_3 E_3$. For $R = -In$, a bivector with coordinates n, we need the product of three vectors. Splitting $\hat{q}n = \hat{q} \cdot n + \hat{q} \wedge n$ and multiplying by $\hat{p}$, the vector part of $\hat{q}n\hat{p}$ is $(\hat{q}\cdot n)\,\hat{p} + (n\cdot\hat{p})\,\hat{q} - (\hat{q}\cdot\hat{p})\,n$, and its trivector part is the volume $\big(\hat{q}\cdot(n\times\hat{p})\big)I$. Multiplying by $-I$ turns the vector part v into the bivector $-Iv$, whose coordinates are v, and turns the trivector part into the scalar $\hat{q}\cdot(n\times\hat{p}) = n\cdot(\hat{p}\times\hat{q})$. Summing with the weights:

$$\mathcal{K}(-In) = z\cdot n - I\left(Dn + D^T n - (\operatorname{tr} D)\, n\right).$$

So the matrix of $\mathcal{K}$ in the basis $E_0, \ldots, E_3$ is

$$K = \begin{pmatrix} \operatorname{tr} D & z^T \\ z & D + D^T - (\operatorname{tr} D)\, I_{3\times 3} \end{pmatrix}. \tag{3.15}$$

This is Davenport's matrix of the q-method [2,17]. Geometric algebra shows where it comes from: it is the matrix of the operator $R \mapsto \sum_j c_j \hat{q}_j R \hat{p}_j$, which measures the gain. Its trace is $\operatorname{tr} D + 2\operatorname{tr} D - 3\operatorname{tr} D = 0$, so its four eigenvalues add up to zero, and some of them are negative.

3.5. The energy as a sum of squares

The energy has another useful form. For each pair, the map $R \mapsto Rp_j + q_j R$ is linear, and its matrix M_j in the basis (2.6) is a 4×4 real matrix. Expanding the geometric products of bivectors with the inner and commutator products, for $R = w + B$,

$$Rp_j + q_j R = w\,(p_j + q_j) + B \cdot (p_j + q_j) + (q_j - p_j) \times B,$$

and with $s_j = p_j + q_j$ and $d_j = q_j - p_j$ (as 3×1 coefficient vectors),

$$M_j = \begin{pmatrix} 0 & -s_j^T \\ s_j & [d_j]_\times \end{pmatrix}, \tag{3.16}$$

a skew-symmetric matrix, where $[d]_\times$ is the cross-product matrix. Then the energy is $E(R) = \sum_j c_j\, |M_j R|^2 = R^T M R$, with

$$M = \sum_j c_j\, M_j^T M_j = S\, I_{4\times 4} + 2K, \tag{3.17}$$

which agrees with (3.9). Being a sum of matrices of the form $M_j^T M_j$, M is symmetric and positive semi-definite. The next section explains what this positivity really gives us.

4. What Positivity Tells Us

4.1. Bounds on the eigenvalues

It is tempting to conclude that, since the energy is a positive semi-definite quadratic form, Newton's method for its largest eigenvalue is robust. Positivity is useful, but for a different reason. M and K have the same eigenvectors, and their eigenvalues differ by the shift $\lambda = S + 2\mu$; shifting the eigenvalues changes nothing in the eigenvector problem, nor in Newton's method, whose iterates are simply shifted too. What positivity does provide is a *bound*.

There are two sums of squares, one for the sum and one for the difference. Repeating the expansion (3.6) with a minus sign,

$$E_{\pm}(R) = \sum_j c_j \, |Rp_j \pm q_j R|^2 = S \, |R|^2 \pm 2\, G(R) \;\geq\; 0. \tag{4.1}$$

For a unit eigenrotor R_k with eigenvalue μ_k, the gain is $G(R_k) = \mu_k$, and both inequalities give $S \pm 2\mu_k \geq 0$:

$$-\frac{S}{2} \;\leq\; \mu_k \;\leq\; \frac{S}{2} \qquad \text{for every eigenvalue } \mu_k. \tag{4.2}$$

The upper bound is attained exactly when the data are free of noise, since then the difference energy E_- vanishes at the true rotor, which is (3.12). In general, the difference energy at the best rotor is the least squares error L^* of the best rotation, and $E_-(R_0) = S - 2\mu_0$ gives

$$\frac{S}{2} - \mu_0 = \frac{L^*}{2}. \tag{4.3}$$

The bound overestimates the largest eigenvalue by half of the least squares error: it is close to μ_0 exactly when the data can be aligned well. We will use this in Section 5.

4.2. How the eigenvalues depend on the data

To understand the eigenvalues, it helps to look at the data in the coordinate frames where they are simplest. The correlation matrix has a signed singular value decomposition $D = \mathcal{V}\,\mathrm{diag}(s_1, s_2, s_3)\,\mathcal{U}^T$, where $\mathcal{U}$ and $\mathcal{V}$ are rotations, $s_1 \geq s_2 \geq |s_3|$, and s_3 has the sign of $\det D$. In the frame of $\mathcal{U}$ for the $\hat{p}_j$ and the frame of $\mathcal{V}$ for the $\hat{q}_j$, the correlation becomes diagonal. Since, by (3.15), $\mathcal{K}$ depends on the data only through D, we may as well replace the data by three pairs that have the same correlation: the unit vectors e_k of the first frame, paired with $s_k e_k$ in the second frame. In those frames,

$$\mathcal{K}(R) = \sum_{k=1}^{3} s_k \, e_k R \, e_k.$$

Now the eigenrotors can be read off. The identity, $R = 1$, aligns every e_k with itself, and its gain is $s_1 + s_2 + s_3$. The rotation by π about e_1, $R = E_1$, keeps e_1 and reverses e_2 and e_3, so its gain is $s_1 - s_2 - s_3$. Algebraically, $e_1 E_1 e_1 = E_1$ because $E_1 = -Ie_1$ commutes with e_1, while $e_2 E_1 e_2 = -E_1$ because E_1

anticommutes with e_2. The same holds for the other two half-turns. The four eigenvalues are therefore

$$\mu_0 = s_1+s_2+s_3, \quad \mu_1 = s_1-s_2-s_3, \quad \mu_2 = -s_1+s_2-s_3, \quad \mu_3 = -s_1-s_2+s_3, \tag{4.4}$$

and the eigenrotors are the identity and the three half-turns about the principal axes. In the original frames they are $VE_k\widetilde{U}$, where U and V are the rotors of $\mathcal{U}$ and $\mathcal{V}$, and the best rotor is $V\widetilde{U}$: rotate the principal frame of the $\hat{p}_j$ onto that of the $\hat{q}_j$. This is the SVD solution [1], and the signed decomposition automatically avoids reflections.

This picture answers three questions that will come up later.

- *When is the best rotation unique?* When μ_0 is a simple eigenvalue. The smallest gap is $\mu_0 - \mu_1 = 2(s_2 + s_3)$, so the rotation is unique exactly when $s_2+s_3 > 0$. If, for instance, all the $\hat{p}_j$ are parallel, then $s_2 = s_3 = 0$, and every rotation about that direction aligns the data equally well; no method can return a unique answer.
- *What is the characteristic polynomial?* $P(\mu) = \det(\mu-\mathcal{K}) = \prod_k(\mu-\mu_k)$. Expanding with the eigenvalues (4.4): the cubic term vanishes, since the eigenvalues add up to zero; the quadratic coefficient is $-2(s_1^2+s_2^2+s_3^2)$; the linear coefficient is $-8s_1s_2s_3$; and the constant term is the product of the eigenvalues, $\det K$. Since $s_1^2 + s_2^2 + s_3^2 = |D|^2$ (the sum of the squares of the entries) and $s_1s_2s_3 = \det D$,

$$P(\mu) = \mu^4 + a\,\mu^2 + b\,\mu + c, \qquad a = -2\,|D|^2, \quad b = -8\det D, \quad c = \det K. \tag{4.5}$$

- *How fast does P grow at μ_0?* $P'(\mu_0) = \prod_{k\geq 1}(\mu_0-\mu_k) = 8\,(s_1+s_2)(s_1+s_3)(s_2+s_3)$, the product of the three gaps.

5. Optimal Quaternion using 4D Geometric Algebra

The most time consuming task of the estimation is to compute the eigenrotor of $\mathcal{K}$ associated with the largest eigenvalue. In this section and the next we show how to find the largest eigenvalue and its eigenrotor, i.e., the required quaternion, using the 4D geometric algebra $\mathbb{G}_4$.

5.1. The columns as vectors

$\mathcal{K}$ is a linear function on the four-dimensional space $\mathbb{G}_3^+$. We model that space by $\mathbb{R}^4$ with an orthonormal basis $\varepsilon_0, \varepsilon_1, \varepsilon_2, \varepsilon_3$ corresponding to $E_0, \ldots, E_3$, and work in its geometric algebra $\mathbb{G}_4$, with pseudoscalar $J = \varepsilon_0\varepsilon_1\varepsilon_2\varepsilon_3$. In four Euclidean dimensions $J^2 = 1$, so $J^{-1} = J$. Let us define four vectors $\mathbf{m}_0, \ldots, \mathbf{m}_3$, corresponding to the columns of the matrix K:

$$K = \begin{pmatrix} \mathbf{m}_0 & \mathbf{m}_1 & \mathbf{m}_2 & \mathbf{m}_3 \end{pmatrix}, \qquad \mathbf{m}_i = \mathcal{K}(\varepsilon_i). \tag{5.1}$$

The system that we want to solve, $\mathcal{K}(R) = \mu R$, written in its homogeneous form, is

$$\begin{pmatrix} \mathbf{m}_0 - \mu\,\varepsilon_0 & \mathbf{m}_1 - \mu\,\varepsilon_1 & \mathbf{m}_2 - \mu\,\varepsilon_2 & \mathbf{m}_3 - \mu\,\varepsilon_3 \end{pmatrix} \begin{pmatrix} w \\ x \\ y \\ z \end{pmatrix} = 0. \tag{5.2}$$

5.2. The characteristic outer product

The system has a nonzero solution exactly when the matrix in (5.2), the *characteristic matrix*, is singular, that is, when its four columns $\mathbf{m}_i - \mu\,\varepsilon_i$ are linearly dependent. In geometric algebra, linear dependence is expressed by the vanishing of an outer product: four vectors of $\mathbb{R}^4$ are dependent exactly when the oriented volume of the parallelotope they span is zero. So μ is an eigenvalue exactly when the *characteristic outer product* vanishes,

$$(\mathbf{m}_0 - \mu\,\varepsilon_0) \wedge (\mathbf{m}_1 - \mu\,\varepsilon_1) \wedge (\mathbf{m}_2 - \mu\,\varepsilon_2) \wedge (\mathbf{m}_3 - \mu\,\varepsilon_3) = P(\mu)\,J. \tag{5.3}$$

The coefficient of J is the volume, which is the determinant of the characteristic matrix, so it is the characteristic polynomial $P(\mu)$ [8].

We will evaluate P several times, once per Newton step, so it pays to compute its coefficients once. Expanding the outer product (5.3) symbolically as a function of μ would give a long expression, and evaluating it at every step would repeat the same products of the data over and over. Section 4 gives a shortcut: by (4.5), two of the coefficients come directly from the correlation matrix, and only $c = \det K = P(0)$ needs the four columns. At $\mu = 0$, the outer product (5.3) is the volume of the columns themselves, and it can be computed as the outer product of two bivectors,

$$c\,J = (\mathbf{m}_0 \wedge \mathbf{m}_1) \wedge (\mathbf{m}_2 \wedge \mathbf{m}_3). \tag{5.4}$$

Each of the two bivectors has six coefficients, the 2×2 minors of a pair of columns, and the outer product combines them into a single number (equation (7.5) below). After that, evaluating $P(\mu)$ and $P'(\mu)$ costs a few multiplications.

5.3. Finding the largest eigenvalue

A simple way to find the largest root of P is Newton's method, $\mu^{(t+1)} = \mu^{(t)} - P(\mu^{(t)})/P'(\mu^{(t)})$, which follows the tangent of the graph of P down to the axis. But P has four real roots, and Newton's method converges to one of them that depends on where it starts. Figure 1 shows the danger. If we start between two roots, for instance at $\mu = 1$, as is customary in implementations of QUEST and FOAM, where the weights are normalized so that the largest eigenvalue is close to 1, the iteration can converge to a smaller root and return a wrong rotation. In our tests this happened in about 30% of noisy problems with vectors of random lengths.

The figure also suggests the cure: to the right of the largest root, the graph of P is increasing and convex, so each tangent meets the axis between

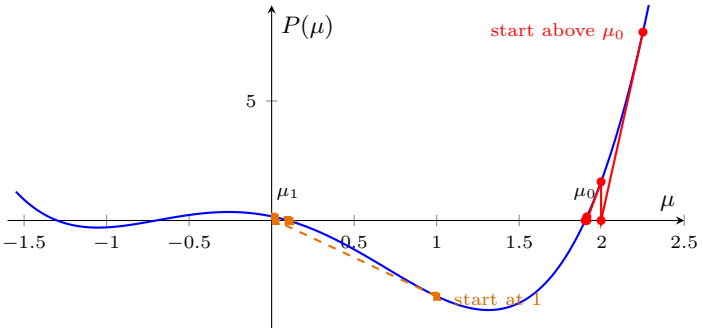


FIGURE 1. Newton's method on a characteristic polynomial with roots $\mu_0 = 1.9$, $\mu_1 = 0.1$, -0.7, -1.3 (singular values $1, 0.6, 0.3$). Started above the largest root (red), every step stays above it and moves toward it. Started at $\mu = 1$ (dashed), between two roots, it converges to μ_1, a wrong eigenvalue.

the current point and the root. Starting there, the iteration slides down monotonically to the right root. All we need is a starting point that is guaranteed to be above the largest root, and the bound (4.2) provides it.

Theorem 5.1. *Starting from $\mu^{(0)} = S/2$, Newton's method on P produces a decreasing sequence $\mu^{(0)} \geq \mu^{(1)} \geq \cdots \geq \mu_0$ that converges to the largest eigenvalue μ_0. For data free of noise it starts at μ_0.*

Proof. $P(\mu) = \prod_k (\mu - \mu_k)$, and for $\mu > \mu_0$ every factor is positive. So are $P'(\mu)$ and $P''(\mu)$, which are sums of products of these factors: P is increasing and convex on $[\mu_0, \infty)$. Suppose $\mu^{(t)} > \mu_0$. Then $P(\mu^{(t)})$ and $P'(\mu^{(t)})$ are positive, so the step is negative and $\mu^{(t+1)} < \mu^{(t)}$. Because P is convex there, its graph lies above the tangent at $\mu^{(t)}$; at μ_0 this says $0 = P(\mu_0) \geq P(\mu^{(t)}) + P'(\mu^{(t)})\,(\mu_0 - \mu^{(t)})$, and dividing by $P'(\mu^{(t)}) > 0$ gives $\mu^{(t+1)} \geq \mu_0$. A decreasing sequence bounded below converges, and its limit is a root of P not smaller than μ_0, hence μ_0 itself. The start $S/2$ is above μ_0 by (4.2), and equal to it without noise by (3.12). □

How many steps are needed? By (4.3), the start overestimates μ_0 by half the least squares error. When the data can be aligned well, the start is very close to the root, and the quadratic convergence of Newton's method near a simple root takes over immediately: one step without noise, two or three with moderate noise. Only for very noisy data, where the least squares error is comparable to S, does the iteration start far from the root, where P behaves like μ^4 and each step shrinks μ by about one quarter. We stop when a step is smaller than a tolerance ϵ (10^{-5} in our implementation). By then the error is of order ϵ^2, relative to the scale of the eigenvalues.

Nothing in this argument is specific to our method. Any method that finds the largest eigenvalue of the same matrix by Newton's method, such as

QUEST, FOAM and FLAE with Newton's method, solves the same quartic up to a shift of μ, and the same start $S/2$, shifted accordingly, makes it converge monotonically to the right root. The start 1 is the special case of unit vectors with weights adding up to one, where $S/2 = 1$; for other data it may lie between two roots.

6. The Eigenrotor as an Intersection of Hyperplanes

6.1. Why intersect hyperplanes

Once μ_0 is known, we need a nonzero solution of the homogeneous system (5.2). Since the characteristic matrix is singular, the system cannot be solved by inverting it, and since every multiple of a solution is also a solution, we have to decide which multiple we want. Following [3], we look at the system geometrically.

Let $\mathbf{x}_i = \mathbf{m}_i - \mu_0 \varepsilon_i$ be the columns of the characteristic matrix, and $r = (w, x, y, z)$ the unit eigenrotor. Because K is symmetric, the columns are also its rows, so each equation of the system says $\mathbf{x}_i \cdot r = 0$: the solution lies on the hyperplane H_i through the origin with normal $\mathbf{x}_i$. The characteristic matrix has rank three, so the four hyperplanes intersect in a line, the line spanned by r. The normals $\mathbf{x}_i$ all lie in the three-dimensional space orthogonal to that line. Finding the eigenrotor means finding the line where the four hyperplanes meet.

6.2. The meet of three hyperplanes

Three hyperplanes in general position already meet in a line, and geometric algebra gives that line directly. For three vectors $\mathbf{a}, \mathbf{b}, \mathbf{c}$ of $\mathbb{R}^4$, the trivector $\mathbf{a} \wedge \mathbf{b} \wedge \mathbf{c}$ represents the three-dimensional space they span, and its dual is the direction orthogonal to it:

$$\mathbf{a} \cdot \big((\mathbf{a} \wedge \mathbf{b} \wedge \mathbf{c})\, J^{-1}\big) = (\mathbf{a} \wedge \mathbf{a} \wedge \mathbf{b} \wedge \mathbf{c})\, J^{-1} = 0, \tag{6.1}$$

and likewise for $\mathbf{b}$ and $\mathbf{c}$. So $(\mathbf{a} \wedge \mathbf{b} \wedge \mathbf{c})J^{-1}$ lies on the three hyperplanes with normals $\mathbf{a}, \mathbf{b}, \mathbf{c}$: it is their *meet*. Its length is the three-dimensional volume of the parallelotope spanned by $\mathbf{a}, \mathbf{b}, \mathbf{c}$, because multiplying by J^{-1} preserves norms. The meet is long when the three normals are well spread out, and it shrinks to zero when they become coplanar, which is exactly when the three hyperplanes stop meeting in a single line.

With four normals there are four ways of choosing three. Leaving out one column at a time, define the four trivectors

$$T_0 = \mathbf{x}_1 \wedge \mathbf{x}_2 \wedge \mathbf{x}_3, \quad T_1 = \mathbf{x}_0 \wedge \mathbf{x}_2 \wedge \mathbf{x}_3, \quad T_2 = \mathbf{x}_0 \wedge \mathbf{x}_1 \wedge \mathbf{x}_3, \quad T_3 = \mathbf{x}_0 \wedge \mathbf{x}_1 \wedge \mathbf{x}_2. \tag{6.2}$$

Each meet $T_k J^{-1}$ lies on the line of r, so it is a multiple of r. Which multiple? The answer is the key to everything that follows.

6.3. How long is each meet

Think of the characteristic matrix as a linear function $\mathcal{G} = \mathcal{K} - \mu_0$ on $\mathbb{R}^4$. It maps each basis vector ε_i to the column $\mathbf{x}_i$, and it maps the eigenrotor r to zero: it flattens the direction r and keeps the three directions orthogonal to it. Now, the three columns in T_k are the images of the three basis vectors other than ε_k, which together span the coordinate hyperplane $\varepsilon_k^\perp$ (the elements whose k-th coordinate is zero). So T_k is the image under $\mathcal{G}$ of the unit volume of $\varepsilon_k^\perp$, and its size tells us how much $\mathcal{G}$ shrinks volumes in that hyperplane. If $\varepsilon_k^\perp$ contains the flattened direction r, that is, if $r_k = 0$, its image is squashed onto a plane and the volume is zero. If $\varepsilon_k^\perp$ is orthogonal to r, which happens when $r = \pm\varepsilon_k$, nothing in it is flattened. In between, the volume is proportional to how far $\varepsilon_k^\perp$ is from containing r, which is $|\cos\angle(r, \varepsilon_k)| = |r_k|$. The precise statement is:

Proposition 6.1. *Let* $\kappa = -P'(\mu_0) = -8\,(s_1+s_2)(s_1+s_3)(s_2+s_3)$. *Then*

$$T_k\, J^{-1} = (-1)^k\, \kappa\, r_k\, r, \qquad k = 0, 1, 2, 3. \tag{6.3}$$

Proof. A linear function extends to multivectors as an outermorphism, $\mathcal{G}(\mathbf{a}\wedge\mathbf{b}\wedge\mathbf{c}) = \mathcal{G}(\mathbf{a})\wedge\mathcal{G}(\mathbf{b})\wedge\mathcal{G}(\mathbf{c})$ [8], so $T_k = \mathcal{G}(\hat{\varepsilon}_k)$, where $\hat{\varepsilon}_k$ is the outer product of the basis vectors other than ε_k, in increasing order. Moving ε_k through the others shows $\hat{\varepsilon}_k = (-1)^k\,\varepsilon_k J$; for example $\varepsilon_1 J = \varepsilon_1\varepsilon_0\varepsilon_1\varepsilon_2\varepsilon_3 = -\varepsilon_0\varepsilon_2\varepsilon_3$. Hence $T_k J^{-1} = (-1)^k\,\mathcal{G}(\varepsilon_k J)J^{-1}$. For a symmetric $\mathcal{G}$, the map $\mathbf{a} \mapsto \mathcal{G}(\mathbf{a}J)J^{-1}$ is its adjugate, which satisfies $\mathcal{G}\big(\mathcal{G}(\mathbf{a}J)J^{-1}\big) = \det(\mathcal{G})\,\mathbf{a}$ [4,8]. Here $\det\mathcal{G} = 0$, so the adjugate maps every vector into the kernel of $\mathcal{G}$, the line of r; being symmetric, it is $\mathbf{a} \mapsto \kappa\,(r\cdot\mathbf{a})\,r$ for some number κ, which is its trace: the sum of the principal 3×3 minors of $\mathcal{G}$, or equivalently the product of its three nonzero eigenvalues $\mu_k - \mu_0$, $k = 1, 2, 3$. By Section 4, this is $-P'(\mu_0)$, the product of the three gaps with a minus sign. □

So every meet points along the eigenrotor, and its length $|\kappa|\,|r_k|$ has two factors. The first one, $|\kappa|$, is the product of the three gaps between μ_0 and the other eigenvalues; it depends only on the data, and it vanishes only when the best rotation is not unique. The second one, $|r_k|$, is the size of the k-th coordinate of the eigenrotor, and it depends on the rotation. A meet is a reliable way of computing the eigenrotor when r_k is large, and a useless one when r_k is close to zero: then it is a tiny vector made mostly of rounding errors.

6.4. A tempting alternative: constraining all the hyperplanes

Since each single meet fails when one coordinate of the eigenrotor vanishes, a natural idea is to treat the four hyperplanes alike. Instead of fixing the scale of the solution through one of them, we could add the same offset γ to all four and intersect them, $(\mathbf{x}_0+\gamma)\wedge(\mathbf{x}_1+\gamma)\wedge(\mathbf{x}_2+\gamma)\wedge(\mathbf{x}_3+\gamma)$. Distributing the outer product, the terms of grade three are γ times the sum of the four trivectors (6.2), so the solution would be $\sum_k T_k J^{-1}$. The idea is appealing: it is symmetric, it costs no more than a single meet, and it seems to avoid

relying on any particular coordinate. Proposition 6.1 lets us check whether it does, by telling us exactly what the sum computes:

$$\sum_{k=0}^{3} T_k \, J^{-1} = \kappa \, (r_0 - r_1 + r_2 - r_3) \, r. \tag{6.4}$$

The four meets point along the same line, but with signed lengths κr_0, $-\kappa r_1$, κr_2, $-\kappa r_3$, and they cancel when $r_0 - r_1 + r_2 - r_3 = 0$. This happens for many common rotations: for example the rotation by $\pi/2$ about the z axis, whose coordinates are $(\frac{1}{\sqrt{2}}, 0, 0, \frac{1}{\sqrt{2}})$, or the rotation by $2\pi/3$ about the axis $(1,1,1)/\sqrt{3}$, whose coordinates are $(\frac{1}{2}, \frac{1}{2}, \frac{1}{2}, \frac{1}{2})$. Constraining all the hyperplanes does not remove the singularities; it moves them to a different family of rotations. The reason is that the sum, too, is decided before looking at the data: it is one more fixed combination of the meets, and the next section shows that every fixed combination fails somewhere.

7. Optimal Computation

7.1. The matrix

The matrix K of (3.15) and the coefficients of P depend on the data only through the correlation matrix D, a 3×3 matrix, and the scalar S. Both are accumulated in a single pass over the data, $D = \sum_j c_j \, \hat{q}_j \hat{p}_j^T$ and $S = \sum_j c_j \, (|\hat{p}_j|^2 + |\hat{q}_j|^2)$, which is the only part of the computation whose cost grows with the number of vectors. Every other step has a fixed cost. The vector z of (3.14) is the antisymmetric part of D: $z = (D_{32} - D_{23}, \, D_{13} - D_{31}, \, D_{21} - D_{12})$.

7.2. The meets

All four meets can be formed from two bivectors,

$$A = \mathbf{x}_0 \wedge \mathbf{x}_1, \qquad C = \mathbf{x}_2 \wedge \mathbf{x}_3, \tag{7.1}$$

since a bivector and a vector commute under the outer product:

$$T_0 = \mathbf{x}_1 \wedge C, \qquad T_1 = \mathbf{x}_0 \wedge C, \qquad T_2 = A \wedge \mathbf{x}_3, \qquad T_3 = A \wedge \mathbf{x}_2. \tag{7.2}$$

In coordinates, a bivector has the six coefficients A_{ij}, $i < j$, on $\varepsilon_i \wedge \varepsilon_j$, and a trivector the four coefficients $T_{012}, T_{013}, T_{023}, T_{123}$. The products we need are

$$(\mathbf{a} \wedge \mathbf{b})_{ij} = a_i b_j - a_j b_i, \tag{7.3}$$

$$(A \wedge \mathbf{x})_{ijk} = A_{ij} \, x_k - A_{ik} \, x_j + A_{jk} \, x_i, \qquad i < j < k, \tag{7.4}$$

$$(A \wedge C) \, J^{-1} = A_{01}C_{23} - A_{02}C_{13} + A_{03}C_{12} + A_{12}C_{03} - A_{13}C_{02} + A_{23}C_{01}, \tag{7.5}$$

$$T J^{-1} = T_{123} \, \varepsilon_0 - T_{023} \, \varepsilon_1 + T_{013} \, \varepsilon_2 - T_{012} \, \varepsilon_3. \tag{7.6}$$

Equation (7.5) is also how the coefficient c of (5.4) is computed, from the bivectors of the columns $\mathbf{m}_i$.

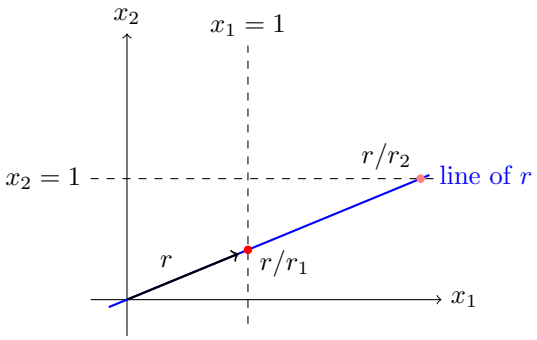


FIGURE 2. Fixing the scale of a line, in two dimensions. Fixing $x_1 = 1$ selects the point r/r_1, close to the origin here. Fixing $x_2 = 1$ selects r/r_2, which moves to infinity as the line becomes parallel to $x_2 = 1$ ($r_2 \to 0$). The coordinate with the largest $|r_k|$ always gives a nearby point.

8. Robustness and Singularities

8.1. Every fixed choice fails somewhere

The eigenrotor is a line; to compute it, every method has to fix its scale somehow. Choosing the meet $T_k J^{-1}$ fixes it in proportion to r_k; choosing the sum (6.4) fixes it in proportion to $r_0 - r_1 + r_2 - r_3$. In general, a method that fixes a linear combination $\sigma \cdot r$ of the coordinates in advance computes the point where the line meets the hyperplane $\sigma \cdot \mathbf{x} = 1$, the point $r/(\sigma \cdot r)$. Figure 2 shows the situation in two dimensions: when the line becomes parallel to the chosen hyperplane, the intersection point moves to infinity and the computation breaks down.

No fixed choice of σ can avoid this. The eigenrotors with $\sigma \cdot r = 0$ are the intersection of the unit sphere of rotors with a hyperplane through its center, a great two-sphere, which is never empty: every fixed choice fails on a two-parameter family of rotations. Near that family it loses accuracy gradually, since the computed meet is proportional to $\sigma \cdot r$ while its rounding errors are not; the relative error grows like $u/|\sigma \cdot r|$, where u is the unit roundoff. The closed form methods fall in this category:

- QUEST [17] computes the eigenrotor from a closed formula whose scalar part γ is the scale, the choice r_0. It fails when the scalar part of the rotor vanishes, that is, for every rotation by π, since the scalar part of a rotor is $\cos \frac{\theta}{2}$.
- FLAE [22] and the Symbolic method [21] fix the last coordinate of the quaternion, the choice r_3. They fail when $r_3 = \sin \frac{\theta}{2} n_3 = 0$: for every rotation about an axis orthogonal to the z axis, and for the identity.
- The sum of the four meets of Section 6.4 is the choice $\sigma = (1, -1, 1, -1)$ and fails on the family described there.

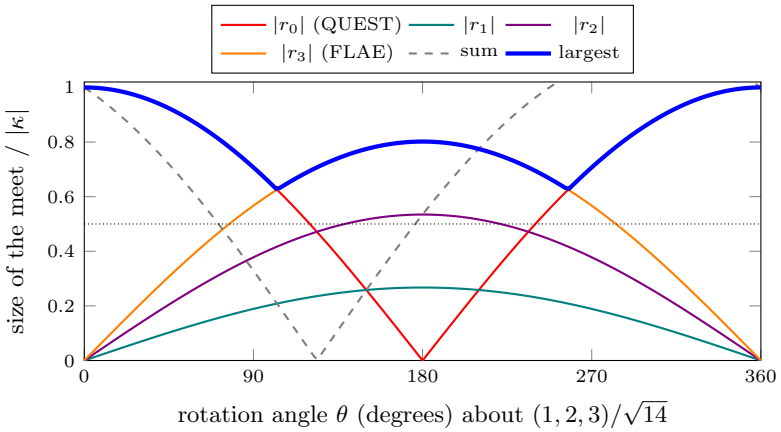


FIGURE 3. Size of each meet, divided by $|\kappa|$, for the rotations by θ about a fixed axis. The meet $T_0 J^{-1}$ (as in QUEST) vanishes at $\theta = \pi$; the meets T_1, T_2, T_3 (the last one as in FLAE) vanish at the identity; the sum of the four meets vanishes near $\theta = 0.7\pi$. The largest meet never goes below $\frac{1}{2}$ (dotted line).

Figure 3 follows the size of each candidate as a rotation turns from 0 to 2π about a fixed axis. Every single meet, and the sum, reaches zero somewhere. But the four coordinates of a unit eigenrotor cannot all be small at once: their squares add up to one, so the largest one always satisfies $|r_k| \geq \frac{1}{2}$.

8.2. Choosing the hyperplanes from the data

The cure is to let the data choose: use the meet with the largest $|r_k|$. We do not know r before computing it, but Proposition 6.1 tells us how to find the best meet anyway. The component of the meet $T_k J^{-1}$ along its own missing direction ε_k is

$$\varepsilon_k \cdot (T_k J^{-1}) = (-1)^k \, \kappa \, r_k^2, \tag{8.1}$$

which is largest, in absolute value, for the largest $|r_k|$. This component is a single coefficient of T_k: by (7.6), it is $T_{0,123}$, $-T_{1,023}$, $T_{2,013}$ or $-T_{3,012}$ for $k = 0, 1, 2, 3$, each a sum of three products by (7.4). So we compute the four coefficients from the bivectors A and C, pick the largest, and complete and dualize only that trivector. Since the largest $|r_k|$ is at least $\frac{1}{2}$, the selected meet satisfies

$$|T_{k^*} J^{-1}| = |\kappa| \, |r_{k^*}| \;\geq\; \frac{|\kappa|}{2} = 4\,(s_1 + s_2)(s_1 + s_3)(s_2 + s_3). \tag{8.2}$$

The right-hand side depends only on the singular values of the data, not on the rotation, and it vanishes only when the best rotation is not unique. Whatever the rotation, the selected meet is computed with a relative error

of the order of the unit roundoff. The remaining degeneracy belongs to the problem, not to the method.

The same idea appears elsewhere. Shepperd's method [16] converts a rotation matrix into a quaternion by first finding the largest of the four components, from the diagonal of the matrix, and solving for the others relative to it. QUEST can be made robust with the method of sequential rotations [17]: when γ is too small, the problem is solved again in a reference frame rotated by π about a coordinate axis, which exchanges the roles of the four coordinates. In our implementation of FLAE, when the leading 3×3 minor of $K - \mu_0 I$ shows that $q_z^2 < 0.01$, the eigenrotor is read instead from the column of the adjugate of $K - \mu_0 I$ with the largest diagonal entry, which is the selection of this section written with matrices. In the language of this section, all of them select a better hyperplane. Here the selection costs almost nothing, because the four candidates come from the same two bivectors.

9. Algorithms

The pseudo-code of the proposed method is shown in Algorithm 1. After the loop over the data, it needs about 40 multiplications to form K and the coefficients of P, about 10 per Newton step, and about 40 to form the two bivectors, the four selection coefficients and the selected trivector. An optimized C++ implementation using the Eigen library [7], a port to Rust, and the experiments of the next section are publicly available on GitHub [11].

10. Comparisons

We selected several representative methods for comparison: FLAE [22] (with its closed formula and with Newton's method), the Symbolic method [21], FA3R [20], QUEST and the Q-Method [17], FOAM [12], Horn's method [9], and SVD [1], both with Eigen's SVD and with a fast 3×3 SVD. The Eigen library [7] was employed to implement all of them, in plain scalar code: no method is vectorized by hand, so that differences in speed reflect the methods and not the effort spent on each implementation. The tests ran on an Apple M1 Pro, compiled with Apple Clang 21 and the options `-O3` and `-Ofast`.

To compare the methods on equal terms, every implementation must reach the optimum on the data it is given. We therefore made a few changes to the implementations as they are usually written: the Newton iterations of QUEST, FOAM and FLAE start at $S/2$ instead of 1, which Theorem 5.1 justifies for them as well and which matters for vectors that are not of unit length; the SVD method corrects reflections, which it can otherwise return for planar or noisy data; FA3R normalizes the correlation matrix first, since its iteration is tuned for data of unit scale; and the fast 3×3 SVD uses the correct entry when sorting the singular values and enough sweeps for double precision. The implementations of FLAE and QUEST used for timing include

Algorithm 1 Robust Rotor Estimation. The result $r/|r|$ is the quaternion (w, x, y, z) of the rotor $R = w + xE_1 + yE_2 + zE_3$.

Require: $P = \{p_j\}_{j=1}^n$, $Q = \{q_j\}_{j=1}^n$, $C = \{c_j\}_{j=1}^n$, as vectors (the duals of the bivectors)

$S \leftarrow \sum_j c_j \left(|p_j|^2 + |q_j|^2\right), \quad D \leftarrow \sum_j c_j\, q_j p_j^T$

$t \leftarrow \operatorname{tr} D, \quad z \leftarrow (D_{32} - D_{23},\, D_{13} - D_{31},\, D_{21} - D_{12})$

$N \leftarrow D + D^T - t\, I$

$\mathbf{m}_0 \leftarrow (t, z), \quad \mathbf{m}_i \leftarrow (z_i, N_{1i}, N_{2i}, N_{3i}),\ i = 1, 2, 3$ (3.15)

$a \leftarrow -2|D|^2, \quad b \leftarrow -8 \det D$ (4.5)

$c \leftarrow (\mathbf{m}_0 \wedge \mathbf{m}_1) \wedge (\mathbf{m}_2 \wedge \mathbf{m}_3)\, J^{-1}$ (5.4) {Newton-Raphson from the upper bound}

$\mu \leftarrow S/2$

repeat

$$\mu \leftarrow \mu - \frac{\mu^4 + a\mu^2 + b\mu + c}{4\mu^3 + 2a\mu + b}$$

until the step is smaller than ϵ {the meets}

$\mathbf{x}_i \leftarrow \mathbf{m}_i - \mu\, \varepsilon_i,\ i = 0, \ldots, 3$

$A \leftarrow \mathbf{x}_0 \wedge \mathbf{x}_1, \quad C \leftarrow \mathbf{x}_2 \wedge \mathbf{x}_3$ (7.1)

$o_0 \leftarrow |(\mathbf{x}_1 \wedge C)_{123}|, \quad o_1 \leftarrow |(\mathbf{x}_0 \wedge C)_{023}|$ (8.1)

$o_2 \leftarrow |(A \wedge \mathbf{x}_3)_{013}|, \quad o_3 \leftarrow |(A \wedge \mathbf{x}_2)_{012}|$

$k^* \leftarrow \arg\max_k o_k, \quad r \leftarrow T_{k^*} J^{-1}$ (7.2), (7.6)

return $r/|r|$

safeguards against their singularities, which do not trigger on the random data used for timing.

10.1. Singular cases

We first isolate the effect of the choice of hyperplanes. We use a single implementation of Algorithm 1 and change only its last step: the meet $T_0 J^{-1}$ (the choice of QUEST), the meet $T_3 J^{-1}$ (the choice of FLAE), the sum of the four meets (Section 6.4), and the selected meet. The input is 1000 unit vectors with random directions, rotated without noise. Table 1 shows the angle between the estimated and the true rotation. Each fixed choice fails, with errors of the order of a radian, exactly on the rotations predicted in Section 8, and is accurate elsewhere. The selected meet is as accurate as Eigen's eigensolver applied to the same matrix in every case, and Newton's method needed a single step in all of them, as predicted by Theorem 5.1.

Figure 4 shows what happens near the singular families. On the left, the rotations are by $\pi - \delta$, so that $r_0 = \sin\frac{\delta}{2}$ approaches zero; on the right, they are by $\pi/2 + \delta$ about the z axis, approaching the family of the sum. The error of the affected choice grows like u/δ, as predicted, while the selected meet stays at the level of rounding errors.

TABLE 1. Angle error (radians) for noise-free data, with different choices of hyperplanes. The coordinates (w, x, y, z) of the true rotors are, from top to bottom, $(1, 0, 0, 0)$, $(0, 0.27, 0.53, 0.80)$, $(0.71, 0, 0, 0.71)$, $(0.5, 0.61, 0.61, 0)$ and $(0.88, 0.13, 0.26, 0.38)$.

Rotation	T_0 (QUEST)	T_3 (FLAE)	sum of meets	selected	eigen-solver
identity	$6.4 \cdot 10^{-18}$	3.07	$2.9 \cdot 10^{-16}$	$6.4 \cdot 10^{-18}$	0
π about $(1, 2, 3)$	1.46	$3.7 \cdot 10^{-16}$	$3.0 \cdot 10^{-16}$	$3.7 \cdot 10^{-16}$	$2.1 \cdot 10^{-16}$
$\pi/2$ about e_3	$7.6 \cdot 10^{-18}$	$2.2 \cdot 10^{-16}$	0.69	$7.6 \cdot 10^{-18}$	$1.0 \cdot 10^{-16}$
$2\pi/3$ about $(1, 1, 0)$	$3.3 \cdot 10^{-16}$	0.67	$5.4 \cdot 10^{-16}$	$3.3 \cdot 10^{-16}$	$6.6 \cdot 10^{-16}$
1 rad about $(1, 2, 3)$	$5.0 \cdot 10^{-16}$	$5.1 \cdot 10^{-16}$	$9.8 \cdot 10^{-16}$	$5.0 \cdot 10^{-16}$	$6.3 \cdot 10^{-16}$

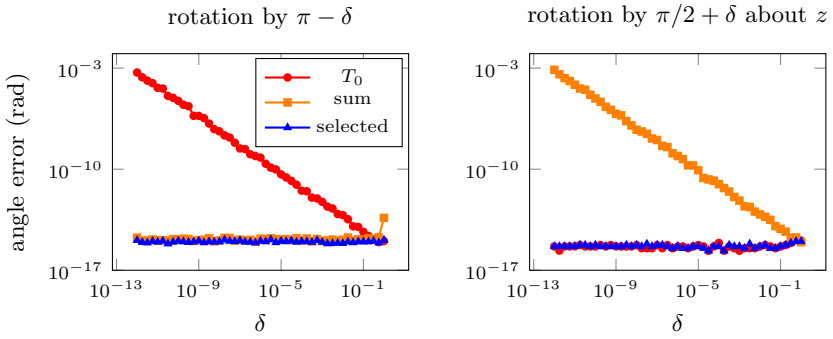


FIGURE 4. Largest angle error over 20 noise-free problems of 1000 unit vectors, approaching the singular family of T_0 (left) and of the sum of the four meets (right).

10.2. Accuracy

Next we test the library implementation of Algorithm 1 on 100 000 random problems: between 3 and 1000 vectors (log-uniformly), random rotations, random weights in $[0.1, 1]$, half of the problems with unit vectors and half with lengths in $[1, 1000]$, and a fifth of them with all the p_j in a plane. Each target is $q_j = Rp_j\widetilde{R} + \eta\,|p_j|\,g_j$ with Gaussian vectors g_j, for five noise levels η. We compare with the eigenvector given by Eigen's eigensolver for the same matrix. Table 2 shows the worst case over 20 000 problems per noise level. The error of the estimate never exceeds that of the eigensolver by more than a few parts in 10^9, and without noise the rotation is recovered to $3 \cdot 10^{-11}$ radians. The angle between the two solutions can be larger, up to $6 \cdot 10^{-5}$ radians for $\eta = 0.1$, in problems where the gap $2(s_2 + s_3)$ is small, for example planar data with few vectors: there, quite different rotations have almost the same error, and the rotation is poorly determined by the data, whichever method is used. The number of Newton steps grows with the noise, as explained in

TABLE 2. Worst cases of Algorithm 1 over 20 000 random problems per noise level η, compared with an eigensolver: angle between the two solutions, relative excess of the least squares error, angle to the true rotation (without noise), and number of Newton steps.

η	angle to eigensolver	excess error	angle to truth	mean steps	max steps
0	$3.4 \cdot 10^{-11}$	–	$3.3 \cdot 10^{-11}$	1.00	1
$1 \cdot 10^{-6}$	$1.2 \cdot 10^{-10}$	$5.6 \cdot 10^{-9}$	–	1.27	2
$1 \cdot 10^{-3}$	$1.6 \cdot 10^{-8}$	$5.2 \cdot 10^{-12}$	–	2.23	3
0.1	$5.8 \cdot 10^{-5}$	$6.6 \cdot 10^{-11}$	–	4.04	9
1	$2.2 \cdot 10^{-8}$	$3.5 \cdot 10^{-15}$	–	8.60	16

Section 5: one step without noise, two or three for moderate noise, and up to 16 when the noise is as large as the data.

10.3. Speed

Table 3 and Figure 5 compare the time per estimation for n vectors. For each n we use a pool of 256 problems with uniformly random rotations and small noise, so that data-dependent branches are exercised as they would be in practice, and report the best of five runs over the pool. For small n, Algorithm 1 is among the fastest methods, together with QUEST, FOAM and FLAE with Newton's method, all of which solve the same quartic with a few Newton steps. The differences among these four are within about 20% and change with n and the compiler options: QUEST is the fastest for $n = 3$, FOAM for n between 10 and 100 with `-O3`, and Algorithm 1 for $n = 10$ with `-Ofast`. Methods that call a general eigensolver or SVD are 11 to 19 times slower for $n = 3$. For large n, the loop over the data dominates every method, and the methods that need only D (FLAE with its closed formula, Symbolic, FA3R) take 30–40% less time than Algorithm 1 at $n = 1000$ with `-O3`, and 20–30% less than QUEST, since they do not accumulate S. With `-Ofast`, which allows the compiler to reorder the sums, these differences shrink to 10% or less. The compiler options also change the ranking of other methods: with `-Ofast` the fast 3×3 SVD becomes slower for large n, and the Q-Method is faster than Algorithm 1 at $n = 1000$ with `-O3` but not with `-Ofast`. Comparisons of speed should therefore state the options used.

At this level, small details of an implementation matter as much as the method. For example, writing the loop over the data with explicit sums instead of Eigen expressions changes the time of a method by up to 15%. The next subsection therefore compares the proposed method and QUEST, the two fastest robust methods for small n, with implementations written in exactly the same style.

TABLE 3. Time per estimation (ns) for n vectors. The fastest method for each n and compiler option is in bold.

	`-O3`				`-Ofast`			
Method	3	10	100	1000	3	10	100	1000
Proposed	40	64	271	2250	**32**	**51**	202	1632
QUEST	36	60	246	1954	33	54	203	1516
FOAM	**35**	**58**	**238**	2140	47	67	**192**	1704
FLAE Newton	44	72	251	2018	39	56	200	1551
FLAE	226	258	406	1622	163	182	319	1609
Symbolic	67	81	245	**1449**	60	74	230	**1507**
FA3R	205	224	359	1592	159	171	297	1613
FA3R Int	132	146	262	1513	136	148	264	1598
Q-Method	742	780	847	2050	718	735	827	2111
Horn	750	783	878	2269	716	738	832	2227
SVD	606	680	1033	4514	582	654	1009	4606
SVD McAdams	469	482	642	1972	463	490	747	3090
GA Valkenburg	791	856	1530	8785	755	808	1456	8340

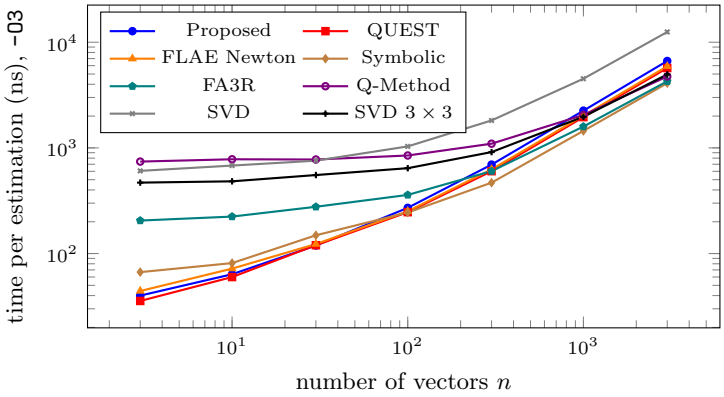


FIGURE 5. Time per estimation as a function of the number of vectors, compiled with `-O3`.

10.4. Solving many problems at once with SIMD

Many applications estimate a large number of independent rotations: one per element in shape deformation, one per frame or per sensor in tracking. Such problems can be solved several at a time with SIMD instructions, which apply the same operation to several numbers at once: each lane of a vector register holds a different problem. This works well only when all lanes execute the same instructions. A data-dependent branch, a test whose outcome differs between problems, forces all the lanes to execute both paths whenever any lane needs the other one.

We compare the four methods of Table 3 that solve the quartic with Newton's method, since they are the fastest for small n, and they differ in their branches. The selection of the meet in Algorithm 1 has no branch: all four candidates come from the same two bivectors A and C, and choosing the largest is a comparison followed by conditional selects, which are SIMD operations. FOAM has no branch either: after Newton's method, a single formula gives the rotation matrix. When a quaternion is needed, it is read from the matrix as Shepperd does [16]: the columns of $4\,\mathbf{q}\mathbf{q}^T$ are linear in the entries of the matrix, and the one with the largest diagonal entry, at least 1, is chosen with conditional selects, the same choice as that of the meet. FLAE as published has no branch, but it fixes $q_z = -1$ and fails for rotations about horizontal axes (Section 8). With the fallback of Section 8, which reads the eigenrotor from the best column of the adjugate when $q_z^2 < 0.01$, it is robust but has a branch, taken for about 13% of random rotations. The robust version of QUEST also has a branch: when $q_w^2 = \gamma / P'(\lambda)$ is below a threshold τ, it solves the problem again in a rotated frame (the method of sequential rotations). With $\tau = 0.1$, which keeps the relative error of QUEST of the order of $u/\sqrt{\tau} \approx 3u$, comparable to the bound (8.2) of the proposed method, the branch is taken for about 40% of random rotations (the rotors with $|q_w| < 0.32$). With four lanes, at least one of them needs the branch in $1 - 0.87^4 \approx 42\%$ of the groups for FLAE, and in $1 - 0.6^4 \approx 87\%$ for QUEST. In those groups the branch is computed for all four lanes, and each lane keeps the result it needs.

To compare them on equal terms, we implemented all of them in the same style, with the same accumulation loop, the same data layout, the same Newton iteration from $S/2$ and the same stopping test, once for a single problem and once for four problems in the lanes of Clang's four-wide vector type (two 128-bit NEON registers on our machine). The whole computation runs in the lanes; the only decisions made once per group are whether all the lanes have converged, which ends Newton's method, and whether any lane needs a branch. The data are stored with each coordinate in its own array and the four problems interleaved, the same for every method; in an application, producing that layout is part of the cost. All the implementations agree with the library ones to $2 \cdot 10^{-14}$ radians, except FOAM for $n = 3$, to $2 \cdot 10^{-10}$: its formula for the matrix is exact only at the eigenvalue, so its result depends on where Newton's method stops, and a lane keeps iterating until all the lanes in its group have converged. FLAE as published is compared only where the library does not need its fallback. Tables 4 and 5 show the time per problem, and Tables 6 and 7 the accuracy near the singular sets of QUEST and FLAE.

Among the methods that return a quaternion at full accuracy, the proposed method is the fastest with four lanes: for $n = 3$ it is 7% faster than FOAM followed by the conversion, 20% faster than FLAE with its fallback and 29% faster than QUEST; for $n = 10$ it ties with FOAM and is 14–16% faster than the other two. The gap narrows as n grows, and for $n = 1000$ the loop over the data dominates and all the methods take the same time. The

TABLE 4. Time per problem (ns), `-O3`, four problems at a time in SIMD lanes, for matched implementations, on a pool of 1024 problems with random rotations. The first four methods return a quaternion at full accuracy, and the fastest of them is in bold. The last three trade something: FOAM returns a rotation matrix, FLAE as published fails near $q_z = 0$, and QUEST with $\tau = 10^{-4}$ loses accuracy near rotations by π.

	quaternion, full accuracy				trading something		
n	Proposed	FOAM + quaternion	FLAE + fallback	QUEST $\tau = 0.1$	FOAM matrix	FLAE published	QUEST $\tau = 10^{-4}$
3	**16.7**	18.0	20.9	23.4	12.4	14.0	13.4
10	**26.6**	**26.6**	31.8	30.9	19.5	24.7	21.6
30	**43.7**	44.1	49.1	47.9	37.0	41.6	38.3
100	**103.4**	109.7	116.3	105.8	100.8	107.7	96.3
1000	**1273.2**	1284.6	1285.4	1346.4	1284.4	1277.6	1268.7

TABLE 5. Time per problem (ns), `-O3`, for the implementations of Table 4 run on one problem at a time.

n	Proposed	FOAM + quaternion	FLAE + fallback	QUEST $\tau = 0.1$	FOAM matrix	FLAE published
3	**36.6**	38.5	39.0	37.5	28.5	33.1
10	**54.2**	54.5	54.6	60.1	44.9	50.9
30	**102.6**	**102.6**	107.3	106.1	88.1	104.8
100	**217.0**	220.7	221.8	**217.0**	203.2	224.6
1000	2574.4	2560.2	**2531.4**	2563.0	2552.6	2541.4

TABLE 6. Largest angle error (radians) over 200 noise-free problems of 100 unit vectors, for rotations by $\pi - \delta$ about random axes, whose scalar part is $q_w = \sin\frac{\delta}{2}$.

δ	q_w	Proposed	FOAM	FLAE published	FLAE + fallback	QUEST $\tau = 0.1$	QUEST $\tau = 10^{-4}$
0.1	0.05	$5.6 \cdot 10^{-16}$	$5.4 \cdot 10^{-16}$	$5.2 \cdot 10^{-15}$	$1.8 \cdot 10^{-15}$	$5.4 \cdot 10^{-16}$	$5.1 \cdot 10^{-15}$
0.03	0.015	$4.9 \cdot 10^{-16}$	$4.0 \cdot 10^{-16}$	$7.6 \cdot 10^{-14}$	$1.4 \cdot 10^{-15}$	$5.6 \cdot 10^{-16}$	$1.7 \cdot 10^{-14}$
0.021	0.0105	$6.8 \cdot 10^{-16}$	$4.2 \cdot 10^{-16}$	$5.6 \cdot 10^{-15}$	$1.3 \cdot 10^{-15}$	$6.2 \cdot 10^{-16}$	$2.1 \cdot 10^{-14}$
0.019	0.0095	$5.6 \cdot 10^{-16}$	$4.8 \cdot 10^{-16}$	$4.5 \cdot 10^{-13}$	$1.9 \cdot 10^{-15}$	$7.4 \cdot 10^{-16}$	$7.4 \cdot 10^{-16}$
0.01	0.005	$6.7 \cdot 10^{-16}$	$4.6 \cdot 10^{-16}$	$7.8 \cdot 10^{-15}$	$1.5 \cdot 10^{-15}$	$4.8 \cdot 10^{-16}$	$4.8 \cdot 10^{-16}$
0.001	$5.0 \cdot 10^{-4}$	$5.7 \cdot 10^{-16}$	$4.4 \cdot 10^{-16}$	$7.8 \cdot 10^{-15}$	$2.1 \cdot 10^{-15}$	$5.9 \cdot 10^{-16}$	$5.9 \cdot 10^{-16}$
$1.0 \cdot 10^{-5}$	$5.0 \cdot 10^{-6}$	$5.9 \cdot 10^{-16}$	$4.5 \cdot 10^{-16}$	$1.2 \cdot 10^{-14}$	$2.7 \cdot 10^{-15}$	$5.0 \cdot 10^{-16}$	$5.0 \cdot 10^{-16}$
$1.0 \cdot 10^{-7}$	$5.0 \cdot 10^{-8}$	$6.4 \cdot 10^{-16}$	$4.6 \cdot 10^{-16}$	$1.8 \cdot 10^{-14}$	$1.9 \cdot 10^{-15}$	$5.7 \cdot 10^{-16}$	$5.7 \cdot 10^{-16}$

four lanes make the proposed method 2.2 times faster than its scalar version for $n = 3$, but QUEST only 1.6 times, because its branch is taken in some

TABLE 7. Largest angle error (radians) over 200 noise-free problems of 100 unit vectors, for rotations by random angles in $[0.1, \pi - 0.1]$ about random axes at angle δ from the xy-plane, where q_z is proportional to $\sin\delta$.

δ	Proposed	FOAM	FLAE published	FLAE + fallback	QUEST $\tau = 0.1$
0.1	$5.5 \cdot 10^{-16}$	$3.8 \cdot 10^{-16}$	$3.4 \cdot 10^{-14}$	$5.6 \cdot 10^{-16}$	$5.6 \cdot 10^{-16}$
0.01	$5.7 \cdot 10^{-16}$	$4.6 \cdot 10^{-16}$	$1.2 \cdot 10^{-13}$	$6.1 \cdot 10^{-16}$	$7.8 \cdot 10^{-16}$
0.001	$6.2 \cdot 10^{-16}$	$4.3 \cdot 10^{-16}$	$1.3 \cdot 10^{-12}$	$5.9 \cdot 10^{-16}$	$8.0 \cdot 10^{-16}$
$1.0 \cdot 10^{-5}$	$5.7 \cdot 10^{-16}$	$4.8 \cdot 10^{-16}$	$2.0 \cdot 10^{-10}$	$6.2 \cdot 10^{-16}$	$6.0 \cdot 10^{-16}$
$1.0 \cdot 10^{-7}$	$6.2 \cdot 10^{-16}$	$4.2 \cdot 10^{-16}$	$3.1 \cdot 10^{-8}$	$4.7 \cdot 10^{-16}$	$5.6 \cdot 10^{-16}$
0	$5.6 \cdot 10^{-16}$	$4.3 \cdot 10^{-16}$	3.14	$5.1 \cdot 10^{-16}$	$7.3 \cdot 10^{-16}$

lane in most groups. One problem at a time, the four methods are within 11% of each other for $n \leq 100$.

Faster options exist when something else can be given up. If the rotation matrix is what the application needs, FOAM is the fastest method, 26% faster than the proposed one for $n = 3$ and 27% for $n = 10$: it works with 3×3 matrices and needs no conversion. Its matrix, however, is a rotation only to the precision of Newton's method. FLAE as published is 16% faster than the proposed method for $n = 3$, but its error grows as q_z goes to 0 and reaches π for rotations about horizontal axes (Table 7). QUEST with $\tau = 10^{-4}$ rarely takes its branch and is 20% faster for $n = 3$. Its error near rotations by π, just above the threshold, grows to $2 \cdot 10^{-14}$ radians, 30 times that of the proposed method, in agreement with the $u/|q_w|$ behavior of Section 8. Whether those errors matter depends on the application. The comparison shows that, among the methods that need no compromise, the absence of branches gives the proposed method an advantage when many problems are solved in parallel.

11. An Application: Smooth-Rotation ARAP Deformation

Mesh deformation is a good test for a rotation estimator, because it does not solve one problem but hundreds of thousands, over and over, and because the problems come in a form that is not quite Wahba's. In this section we take a well-known deformation method whose inner loop is a rotation fit computed with an SVD, show that each fit is an instance of the problem of this paper, and measure what changes when the SVD is replaced by Algorithm 1, one cell at a time and four cells at a time with SIMD instructions.

11.1. The deformation model

As-rigid-as-possible (ARAP) deformation [18] moves the vertices x_i of a triangle mesh to new positions x'_i such that the one-ring of every vertex, its *cell*, moves as rigidly as possible. Every cell i has its own rotation R_i, and

the energy measures how far the deformed edges $v_{ij} = x'_j - x'_i$ are from the rotated rest edges $u_{ij} = x_j - x_i$,

$$\sum_i \sum_{j \in \mathcal{N}(i)} w_{ij} \, |R_i u_{ij} - v_{ij}|^2, \tag{11.1}$$

where $\mathcal{N}(i)$ are the d_i neighbors of vertex i and w_{ij} are the cotangent weights of the mesh. With the positions fixed, every cell's rotation can be fitted on its own; with the rotations fixed, the positions solve a linear (Poisson) system. Alternating the two steps, the *local* and the *global* step, decreases the energy. The local step of ARAP is a set of independent Wahba problems, one per cell, usually solved with an SVD.

The rotations of independent cells can change abruptly from one cell to the next, and the deformed surface shows it. Smooth-rotation ARAP (SR-ARAP) [10] adds a bending term that asks neighboring rotations to agree,

$$E = \sum_i \sum_{j \in \mathcal{N}(i)} w_{ij} \, |R_i u_{ij} - v_{ij}|^2 + \alpha A \sum_i \sum_{j \in \mathcal{N}(i)} \frac{1}{d_i} \, \|R_i - R_j\|_F^2, \tag{11.2}$$

where A is the area of the mesh and α a stiffness. Now the rotations are coupled, and the local step is no longer a set of independent problems. It is solved by Gauss–Seidel sweeps: visit the cells one after the other and fit each rotation with its neighbors' current rotations held fixed.

11.2. Each cell update is a Wahba problem

Let us collect the terms of (11.2) that contain one rotation R_i, with everything else fixed. For an edge term, since a rotation preserves lengths,

$$|R_i u - v|^2 = |u|^2 + |v|^2 - 2\, v \cdot (R_i u),$$

and only the last term depends on R_i. For the bending term, $\|R\|_F^2 = 3$ for every rotation, so

$$\|R_i - R_j\|_F^2 = 6 - 2 \operatorname{tr}(R_i^T R_j).$$

The rotation R_i appears in its own bending terms, with weight $1/d_i$, and in the term of each neighbor j that has i among its neighbors, with weight $1/d_j$. Writing $b_{ij} = \alpha A(1/d_i + 1/d_j)$ and dropping the constants, minimizing E over R_i is maximizing

$$\sum_j w_{ij} \, v_{ij} \cdot (R_i u_{ij}) + \sum_j b_{ij} \operatorname{tr}(R_i^T R_j) = \operatorname{tr}(R_i^T M_i),$$

$$M_i = \sum_j w_{ij} \, v_{ij} u_{ij}^T + \sum_j b_{ij} R_j, \tag{11.3}$$

where we used $v \cdot (Ru) = \operatorname{tr}(R^T v u^T)$. This is the orthogonal Procrustes problem for the matrix M_i, which is why SR-ARAP fits each cell with an SVD.

It is also an instance of the problem of this paper. The edge terms are already pairs of vectors: the rest edge u_{ij} should be rotated onto the deformed edge v_{ij}, with weight w_{ij}. The bending terms become pairs too, once we read the trace in a basis: $\operatorname{tr}(R_i^T R_j) = \sum_k (R_j e_k) \cdot (R_i e_k)$. A neighbor's rotation

asks for the three frame vectors e_k to be sent where R_j sends them, with weight b_{ij}. Each cell problem is therefore Wahba's problem with d_i edge pairs and $3d_i$ frame pairs, and its correlation matrix $D = \sum c\,\hat{q}\hat{p}^T$ of Section 9 is exactly M_i. We never form the frame pairs: Algorithm 1 needs only D, and the bound $S/2$ where Newton's method starts.

The bound needs one more thought. The frame pairs contribute $|e_k|^2 + |R_j e_k|^2 = 2$ each, three per neighbor, so $3b_{ij}$ to $S/2$. The edge pairs would contribute $\frac{1}{2}w_{ij}(|u_{ij}|^2 + |v_{ij}|^2)$, but the cotangent weights w_{ij} are negative at obtuse triangles, and the bounds (4.2) were derived from sums of squares with non-negative weights. What Theorem 5.1 really needs is a start above the largest eigenvalue $\mu_0 = s_1 + s_2 + s_3$, and that sum is at most the sum of the singular values of D, its nuclear norm. The nuclear norm of a sum is at most the sum of the nuclear norms, and a single pair has $\|c\,\hat{q}\hat{p}^T\|_* = |c|\,|\hat{p}|\,|\hat{q}| \le \frac{1}{2}|c|(|\hat{p}|^2 + |\hat{q}|^2)$. So

$$\frac{S}{2} = \frac{1}{2}\sum_j |w_{ij}|\left(|u_{ij}|^2 + |v_{ij}|^2\right) + 3\sum_j b_{ij} \tag{11.4}$$

is a valid start, and the proof of Theorem 5.1 applies unchanged. In the middle of a deformation the bound is tight: on the bar of Table 8 below, $S/2$ exceeds μ_0 by 0.01% on average, and Newton's method stops after one step for 10% of the cells, two for 77% and three for the rest.

11.3. Gauss–Seidel in colors, four cells at a time

A Gauss–Seidel sweep is sequential: each cell uses the rotations its neighbors have just received. But it uses only its neighbors. Two cells that are not neighbors can be updated in either order, or at the same time, with exactly the same result. So we color the cells such that no two neighbors share a color, which a greedy pass over the vertices does with 4 to 7 colors on our meshes, and sweep color by color. Within a color the updates are independent, and we solve them four at a time, one cell per SIMD lane. The color order is a different visiting order from the order of the vertex indices, and so a slightly different Gauss–Seidel iteration, but an equally valid one. We use it for every fit, the SVD included, so that all of them compute the same iteration and differ only in how each cell is fitted.

The SR-ARAP update needs rotation matrices: M_i in (11.3) is built from the neighbors' matrices R_j. Algorithm 1 returns a rotor, which we convert to a matrix, and FOAM, which we include for comparison, returns the matrix directly. For the SIMD versions, four cells' matrices M_i and bounds are gathered into the lanes, solved, and the four rotations are scattered back. Everything else in the local step, building M_i from the edges and the neighbors' rotations, is the same for all fits and is not vectorized.

11.4. Results

We implemented the fits in Rust, generic over the lane type, `f64` for one cell and the four-wide `f64x4` of the `wide` crate for four cells (two 128-bit NEON registers), and measured them on the Apple M1 Pro of Section 10, on one

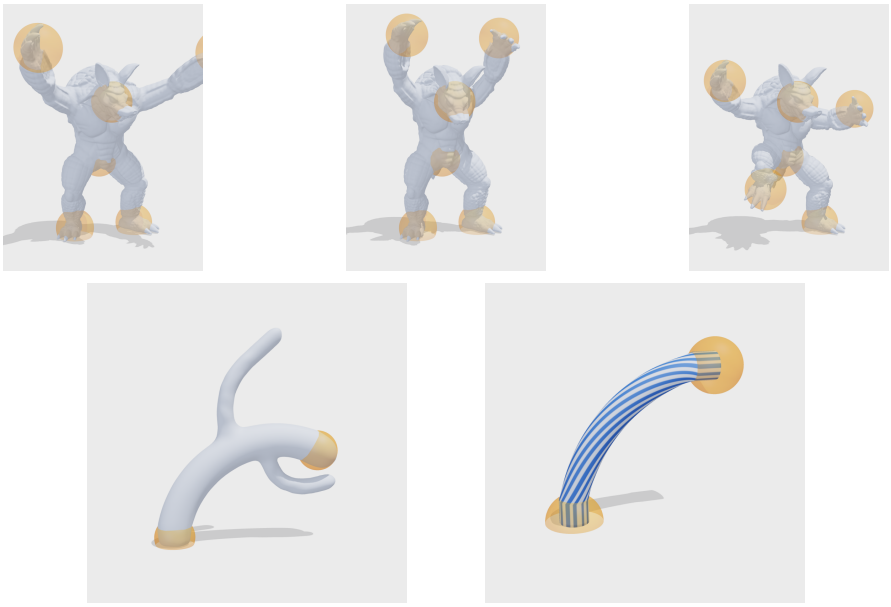


FIGURE 6. SR-ARAP deformations with every cell's rotation fitted by Algorithm 1, four cells at a time in SIMD lanes; the orange spheres are the handles. Top: the Armadillo, solved on its 502-vertex control mesh and transferred to the detailed mesh. Bottom: the Cactus (5,261 cells) with its trunk bent along an arc of 120°, and a bar (7,776 cells) bent into a quarter circle with a half twist, both deformed directly. With the SVD the shapes are the same, to 10^{-15}; one local/global iteration of the first Armadillo pose takes 0.27 ms instead of 0.99 ms.

core. The program is compiled as a single code generation unit: with the default split into several units the compiler did not inline the SIMD code as well, and four lanes lost their advantage. We deform four meshes (Figure 6): the control mesh of the Armadillo used for interactive editing, the Cactus, a bar bent into a quarter circle, and the full Armadillo. After five local/global iterations from the rest pose, we time one local step of three Gauss–Seidel sweeps from the same state with every fit (Table 8), and, separately, the fits alone on the cell problems of that state (Table 9).

The fits agree. After one local step from the same state, the rotations of every method differ from those of the SVD by at most $7 \cdot 10^{-15}$ radians, and after 50 full iterations the vertex positions differ by at most $7 \cdot 10^{-15}$, relative to the half-diagonal of the bounding box. The shapes are the same; only the time changes.

TABLE 8. Time of one SR-ARAP local step (ms), three Gauss–Seidel sweeps in color order, with each cell fitted by the SVD, by Algorithm 1 (proposed), or by FOAM, one cell at a time or four at a time in SIMD lanes. The last column (gain) is the speed-up of the proposed method with SIMD over the SVD.

mesh	cells	colors	SVD	Proposed	Proposed SIMD	FOAM	FOAM SIMD	gain
Armadillo, control	502	6	0.796	0.124	0.108	0.093	0.096	7.4×
Cactus	5,261	6	8.00	1.31	1.15	1.06	1.04	6.9×
Bar	31,274	4	46.5	6.74	5.51	4.84	4.87	8.4×
Armadillo	172,974	7	302.2	74.6	61.0	65.7	57.0	5.0×

TABLE 9. Time of the fit alone (ns per cell) on the cell problems M_i of the same state, stored as one 3×3 matrix per cell, including the conversion of the result to a rotation matrix and, for SIMD, the gathering and scattering of the lanes.

mesh	cells	colors	SVD	Proposed	Proposed SIMD	FOAM	FOAM SIMD	gain
Armadillo, control	502	6	475	42	29	22	24	16.2×
Cactus	5,261	6	452	36	25	20	20	17.8×
Bar	31,274	4	448	39	26	20	19	17.5×
Armadillo	172,974	7	454	38	26	24	20	17.7×

The fit itself becomes 11 to 13 times cheaper with Algorithm 1 one cell at a time, and 16 to 18 times with four lanes, where it costs 25 to 29 ns per cell. The local step, which also builds the matrices M_i, becomes 4 to 7 times faster with one cell at a time and 5 to 8 times with SIMD. Once the fit is cheap, building M_i from the edges and the neighbors' rotations dominates the local step, and that part is the same for every fit; it is also why the full Armadillo, whose 173,000 cells do not fit in the cache, gains less. Four lanes speed up the fit by a factor of about 1.5, less than the 2.2 of Section 10: Newton's method needs only one to three steps here, which leaves less work to share, the matrices have to be gathered into the lanes and scattered back, and every group of four takes as many steps as its slowest cell.

FOAM is faster in this application, by 7 to 14% for the local step. Its fit costs 20 to 24 ns per cell, and four lanes do not make it faster. The reason is the one of Section 10: SR-ARAP needs rotation matrices, which FOAM computes directly from 3×3 matrices, while Algorithm 1 works with 4×4 quantities and returns a rotor that must then be converted. When the rotations are wanted as rotors, for instance to interpolate them across the mesh, the comparison turns the other way. The lesson of the application is

that, once the cell fit is recognized as a Wahba problem with the bound (11.4), any of the Newton methods of this paper turns the SR-ARAP local step from a cost dominated by SVDs into one dominated by bookkeeping, with the same results.

12. Conclusion

We presented a method for estimating the best quaternion aligning two sets of corresponding vectors or bivectors, following the geometry of the problem at each step. Maximizing the length of the sum of each bivector and its rotated target, after multiplying by the rotor, gives an energy that is quadratic in the rotor, and its multivector derivative gives an eigenrotor equation. The positivity of the energies of the sum and of the difference bounds all the eigenvalues by $\pm S/2$, and the upper bound, exact without noise, is the starting point from which Newton's method converges monotonically to the largest eigenvalue. The eigenrotor is the line where the hyperplanes of the characteristic matrix meet, computed with outer products in the 4D geometric algebra.

The same geometric picture explains the singularities of the closed form methods. Each meet of three hyperplanes points along the eigenrotor, with a length proportional to one of its coordinates; any method that fixes in advance which meet, or which combination of meets, to use, fails on a whole family of rotations: rotations by π for QUEST, rotations about horizontal axes and the identity for FLAE, and rotations such as $\pi/2$ about the z axis for the symmetric sum of all four meets. Choosing the meet from the data, by its largest component, removes the singularities, with a guaranteed size that depends only on the data. In our experiments the method always reaches the optimum, at a speed comparable to QUEST, the fastest method for small numbers of vectors. Because the selection of the meet needs no data-dependent branch, the method is also well suited to solving many problems at once with SIMD instructions, where it is the fastest of the methods we tested that return a quaternion at full accuracy, up to 29% faster than QUEST; FOAM is faster still when a rotation matrix is the desired output. In smooth-rotation ARAP deformation, every cell update is a Wahba problem with a tight bound for Newton's method; replacing its SVD by the proposed method makes the local step 5 to 8 times faster with the same results. The selection of the meet does not depend on how the eigenvalue is computed, so it can also make other methods of the same family, such as FLAE, free of singularities.

References

[1] K. S. Arun, T. S. Huang, and S. D. Blostein. Least-squares fitting of two 3-d point sets. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-9(5):698–700, Sep. 1987.

[2] P. B. Davenport. A vector approach to the algebra of rotations with applications. 1968.

[3] S. De Keninck and L. Dorst. Geometric algebra levenberg-marquardt. In M. Gavrilova, J. Chang, N. M. Thalmann, E. Hitzer, and H. Ishikawa, editors, *Advances in Computer Graphics*, pages 511–522, Cham, 2019. Springer International Publishing.

[4] C. Doran and A. Lasenby. *Geometric Algebra for Physicists*. Cambridge University Press, 2003.

[5] L. Dorst and J. Lasenby, editors. *Guide to Geometric Algebra in Practice*. Springer, 2011.

[6] D. Eggert, A. Lorusso, and R. Fisher. Estimating 3-d rigid body transformations: a comparison of four major algorithms. *Machine Vision and Applications*, 9(5):272–290, Mar 1997.

[7] G. Guennebaud, B. Jacob, et al. Eigen v3. http://eigen.tuxfamily.org, 2010.

[8] D. Hestenes and G. Sobczyk. *Clifford Algebra to Geometric Calculus: A Unified Language for Mathematics and Physics*. D. Reidel, Dordrecht, 1984.

[9] B. K. P. Horn. Closed-form solution of absolute orientation using unit quaternions. *Journal of the Optical Society of America A*, 4(4):629–642, 1987.

[10] Z. Levi and C. Gotsman. Smooth rotation enhanced as-rigid-as-possible mesh animation. *IEEE transactions on visualization and computer graphics*, 21(2):264–277, February 2015.

[11] M. C. Lopez Belon. Project on github. Available at https://github.com/mauriciocele/fast-rotor-estimation, 2019.

[12] L. Markley. Attitude determination from vector observations: A fast optimal matrix algorithm. *Journal of the Astronautical Sciences*, 41, 07 1993.

[13] L. Markley. 30 years of wahba's problem. 02 1999.

[14] D. Mortari. EULER-q algorithm for attitude determination from vector observations. *Journal of Guidance, Control, and Dynamics*, 21(2):328–334, 1998.

[15] C. B. U. Perwass. *Geometric algebra with applications in engineering*, volume 4 of *Geometry and Computing*. Springer, Berlin; Heidelberg, 2009.

[16] S. W. Shepperd. Quaternion from rotation matrix. *Journal of Guidance and Control*, 1(3):223–224, 1978.

[17] M. D. Shuster and S. D. Oh. Three-axis attitude determination from vector observations. *Journal of Guidance, Control, and Dynamics*, 4(1):70–77, 1981.

[18] O. Sorkine and M. Alexa. As-rigid-as-possible surface modeling. In *Proceedings of the fifth Eurographics symposium on Geometry processing*, SGP '07, pages 109–116, Aire-la-Ville, Switzerland, Switzerland, 2007. Eurographics Association.

[19] G. Wahba. A Least Squares Estimate of Satellite Attitude, 1965.

[20] J. Wu, M. Liu, Z. Zhou, and R. Li. Fast rigid 3d registration solution: A simple method free of svd and eigen-decomposition, 2018.

[21] J. Wu, M. Liu, Z. Zhou, and R. Li. Fast symbolic 3d registration solution, 2018.

[22] J. Wu, Z. Zhou, B. Gao, R. Li, Y. Cheng, and H. Fourati. Fast Linear Quaternion Attitude Estimator Using Vector Observations. *IEEE Transactions on Automation Science and Engineering*, X(X):1–13, 2017.

[23] Y. Yang. Attitude determination using Newton's method on Riemannian manifold. *Proceedings of the Institution of Mechanical Engineers, Part G: Journal of Aerospace Engineering*, 229(14):2737–2742, 2015.

[24] Y. Yang and Z. Zhou. An analytic solution to wahba's problem. *Aerospace Science and Technology*, 30(1):46–49, Oct 2013.

Mauricio Cele Lopez Belon
Madrid, Spain
e-mail: mclopez@outlook.com